

---

# Nhắc lại kiến thức bài trước

*Đa hình - ABC - magic methods - @property - Ngoại lệ*



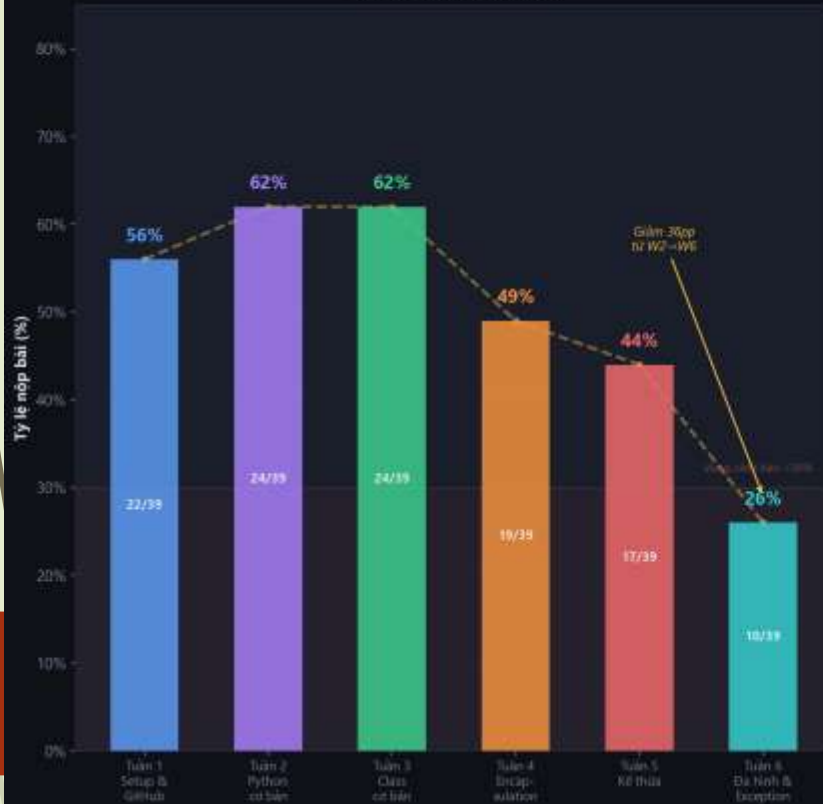
---

# Code Review - Week6

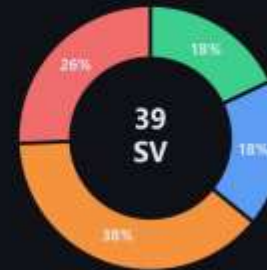
## Báo cáo Tiến độ OOP — Lớp C1 — Thứ 5

OOP - BCSE - VJU - GV: TS. Bùi Huy Kiên - 39 sinh viên - Tuần 1-6

Tỷ lệ nộp bài theo tuần



Phân loại sinh viên



- Nhóm A — W1-W6 đầy đủ: 7
- Nhóm B — Thiếu 1-2 tuần: 7
- Nhóm C — B trễ (W1-W4): 15
- Không có Giới hạn: 10

### SV nổi bật (Nhóm A)

- > Phan Anh Tuấn  
100 LOC, OOP là đây đó, ABC+exception+Phân tích
- > Hoàng Việt Anh  
1000 LOC, Cao nhất lớp - Có cả Tuần 7
- > Lê Văn Thái An  
800 LOC, Chưa phân tích bài
- > Bùi Nhật Minh  
800 LOC, Phân tích bài nhưng thiếu lượng code

### Vấn đề cần lưu ý

- 10/39 SV (26%) không có Github
- File thiếu đuôi: .py: 25/12/23, 25/12/23, 25/12/23
- Viết C++ thay Python: 25/12/24, 25/12/24, 25/12/24

# Code Review - Week6

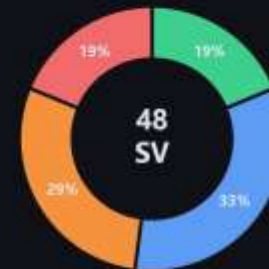
## Báo cáo Tiến độ OOP — Lớp C2 — Thứ 6

OOP - BCSE - VJU - GV: TS. Bùi Huy Kiên - 48 sinh viên - Tuần 1-6

Tỷ lệ nộp bài theo tuần



Phân loại sinh viên



- Nhóm A — W1-W6 đầy đủ: 9
- Nhóm B — Thiếu 1-2 tuần: 16
- Nhóm C — Ít bài (W1-W4): 14
- Không có Github: 9

### SV nổi bật (Nhóm A)

- > **Trần Thu Trang**  
100% LDC, 48 commit - Có LMS, Lattu
- > **Phạm Minh Quý Vương**  
100% LDC, bài tập đầy đủ nhất
- > **Phạm Hồng Duyên**  
100% LDC, bài tập đầy đủ nhất
- > **Trần Ngô Tiến Đạt**  
100% LDC, W3 và Poker game

### Vấn đề cần lưu ý

- 1/48 SV (17%) không có Github + 1 nộp rỗng
- 1 bị lỗi VS Code extension (25/10/2021)
- 1 file mẫu lỗi, py: 25112048, 25112095, 25112015

# Đa hình

## Đa hình (Polymorphism) là gì?

Đa hình (Polymorphism) xuất phát từ tiếng Hy Lạp: poly = nhiều, morph = dạng.

Trong lập trình hướng đối tượng, đa hình là khả năng một phương thức có CÙNG TÊN nhưng thể hiện HÀNH VI KHÁC NHAU tùy thuộc vào đối tượng cụ thể đang gọi nó. Đây là tính chất thứ 4 và cuối cùng trong 4 tính chất OOP.

## Ví dụ thực tế giúp hiểu đa hình

Hãy nghĩ đến hành động "phát ra âm thanh" của các loài động vật:

- Con chó → sủa "Gâu gâu"
- Con mèo → kêu "Meo meo"
- Con vịt → kêu "Quạc quạc"

Cùng một hành động "phát ra âm thanh", nhưng mỗi loài thể hiện KHÁC NHAU.

Đây chính là đa hình: cùng giao diện (interface), khác cài đặt (implementation).

```
# Minh họa bằng Python:  
class Cho:  
    def phat_am(self):  
        return "Gâu gâu"
```

```
class Meo:  
    def phat_am(self):  
        return "Meo meo"
```

```
class Vit:  
    def phat_am(self):  
        return "Quạc quạc"
```

```
# Đa hình: cùng 1 vòng lặp, 3 hành vi  
for con in [Cho(), Meo(), Vit()]:  
    print(con.phat_am())
```

# Đa hình

## Trong Python, đa hình xuất hiện ở 5 dạng:

### 1. Đa hình trong hàm tích hợp

len(), print(),  
abs() ... hoạt  
động với nhiều  
kiểu

### 2. Đa hình trong hàm tự định nghĩa

Hàm ta viết nhận  
được nhiều kiểu  
đối tượng

### 3. Đa hình với các phương thức lớp

Nhiều class cùng  
có phương thức  
giống tên

### 4. Đa hình và kế thừa

Lớp con override  
phương thức lớp  
cha

### 5. Đa hình với 1 hàm và nhiều đối tượng

1 hàm xử lý danh  
sách chứa nhiều  
loại object

# Lớp trừu tượng — Abstract Base Class (ABC)

## Khái niệm Lớp trừu tượng

- Lớp trừu tượng là lớp KHÔNG THỂ tạo đối tượng trực tiếp, chỉ làm "bản thiết kế" cho các lớp con. Nó chứa phương thức trừu tượng (@abstractmethod): chỉ khai báo tên, lớp con BẮT BUỘC override. Và phương thức thường (concrete): có cài đặt đầy đủ, lớp con kế thừa dùng ngay hoặc override.
- Trong Python: kế thừa ABC từ module abc, đánh dấu @abstractmethod. Tương tự interface trong Java.

```
from abc import ABC, abstractmethod

class HìnhHoc(ABC):
    @abstractmethod
    def tinh_dien_tich(self):
        pass
    @abstractmethod
    def tinh_chu_vi(self):
        pass
    def hien_thi(self): # Concrete — dùng ngay
        print(f"S = {self.tinh_dien_tich():.2f}")

# h = HìnhHoc() → TypeError! Không tạo được!

class HìnhTron(HìnhHoc):
    def __init__(self, r): self.r = r
    def tinh_dien_tich(self): # BẮT BUỘC
        return 3.14159 * self.r ** 2
    def tinh_chu_vi(self): # BẮT BUỘC
        return 2 * 3.14159 * self.r
```



## Tại sao cần ABC?

- Ép buộc lớp con override → tránh quên
- Thiết kế "hợp đồng" rõ ràng
- Ngăn tạo đối tượng vô nghĩa (HìnhHoc() trừu tượng quá)
- Kết nối: Trừu tượng hóa + Kế thừa + Đa hình = ABC



## Nếu quên override?

```
class HìnhSai(HìnhHoc):
    def tinh_dien_tich(self): ...
    # thiếu tinh_chu_vi!
hs = HìnhSai() → TypeError!
```



ABC = mắt xích kết nối Trừu tượng hóa + Kế thừa + Đa hình

# Magic Methods — Bảng Tra Nhanh

Implement đúng để đối tượng tích hợp hoàn toàn vào hệ sinh thái Python

| Nhóm     | Method                                         | Khi nào Python gọi?                                 | Trả về                                |
|----------|------------------------------------------------|-----------------------------------------------------|---------------------------------------|
| Chuỗi    | <code>__str__(self)</code>                     | <code>print(obj)</code> , <code>str(obj)</code>     | <code>str</code>                      |
| Chuỗi    | <code>__repr__(self)</code>                    | <code>repr(obj)</code> , trong shell                | <code>str</code>                      |
| Toán tử  | <code>__add__(self, other)</code>              | <code>obj1 + obj2</code>                            | bất kỳ                                |
| Toán tử  | <code>__sub__(self, other)</code>              | <code>obj1 - obj2</code>                            | bất kỳ                                |
| Toán tử  | <code>__mul__(self, other)</code>              | <code>obj1 * obj2</code>                            | bất kỳ                                |
| Toán tử  | <code>__truediv__(self, other)</code>          | <code>obj1 / obj2</code>                            | bất kỳ                                |
| So sánh  | <code>__eq__(self, other)</code>               | <code>obj1 == obj2</code>                           | <code>bool</code>                     |
| So sánh  | <code>__lt__(self, other)</code>               | <code>obj1 &lt; obj2</code> , <code>sorted()</code> | <code>bool</code>                     |
| Sequence | <code>__len__(self)</code>                     | <code>len(obj)</code>                               | <code>int</code>                      |
| Sequence | <code>__getitem__(self, i)</code>              | <code>obj[i]</code>                                 | bất kỳ                                |
| Context  | <code>__enter__</code> / <code>__exit__</code> | <code>with obj as x:</code>                         | <code>self</code> / <code>None</code> |

# Ngoại lệ tích hợp sẵn

Trong Python, có một số ngoại lệ tích hợp có thể được nêu ra khi xảy ra lỗi trong quá trình thực thi chương trình. Các ngoại lệ tích hợp thường gặp:

## Các ngoại lệ tích hợp thường gặp

| Exception         | Ví dụ                      |
|-------------------|----------------------------|
| ValueError        | <code>int("abc")</code>    |
| TypeError         | <code>"abc" + 5</code>     |
| IndexError        | <code>[1,2][5]</code>      |
| KeyError          | <code>d['xyz']</code>      |
| ZeroDivisionError | <code>10 / 0</code>        |
| FileNotFoundError | <code>open('x.txt')</code> |
| AttributeError    | <code>obj.xyz</code>       |



# Custom Exception — Tự thiết kế ngoại lệ

- Python cho phép tự tạo ngoại lệ bằng cách KẾ THỪA từ **Exception**. Tên lỗi rõ ràng theo nghiệp vụ
- (DiemKhongHopLe rõ hơn ValueError), mang thêm dữ liệu (e.diem), phân cấp lỗi theo hệ thống.
- Kết nối OOP: custom exception KẾ THỪA từ Exception, except lớp cha bắt cả lớp con → ĐA HÌNH!

```
# Phân cấp exception theo nghiệp vụ:
class LopHocError(Exception):
    """Gốc cho mọi lỗi hệ thống"""
    pass

class DiemKhongHopLe(LopHocError):
    def __init__(self, diem):
        self.diem = diem
        super().__init__(
            f"Điểm {diem} không hợp lệ (0-10)")

class MaSVTrung(LopHocError):
    def __init__(self, msv):
        self.msv = msv
        super().__init__(f"MSV {msv} đã tồn tại")
```

```
# Sử dụng:
try:
    diem = float(input("Nhập điểm: "))
    if not 0 <= diem <= 10:
        raise DiemKhongHopLe(diem)
except DiemKhongHopLe as e:
    print(f"Lỗi: {e}")
    print(f"Giá trị sai: {e.diem}")
```

```
# Phân cấp → đa hình:
# except LopHocError → bắt CẢ HAI
#   DiemKhongHopLe và MaSVTrung
```

```
# Exception
# └─ LopHocError      ← gốc
#   └─ DiemKhongHopLe ← cụ thể
#   └─ MaSVTrung      ← cụ thể
```

➡ Custom exception = kế thừa + đa hình trong xử lý lỗi.

except LopHocError bắt được cả DiemKhongHopLe lẫn MaSVTrung → đa hình!

# @property — Đóng gói Pythonic

- @property là decorator cho phép truy cập thuộc tính private một cách TỰ NHIÊN như thuộc tính thường, nhưng bên trong vẫn có kiểm soát (validation). Đây là cách Python cài đặt Đóng gói (Encapsulation).
- Thay vì get\_ten() / set\_ten() kiểu Java → dùng obj.ten (getter) và obj.ten = 'A' (setter) kiểu Pythonic.

```
class SinhVien:
    def __init__(self, ten, msv):
        self.__ten = ten        # private
        self.__msv = msv       # private

    @property
    def ten(self):
        """Getter – chỉ đọc, không sửa được"""
        return self.__ten

    @property
    def msv(self):
        return self.__msv

sv = SinhVien("Nguyen Van A", "SV001")
print(sv.ten)        # "Nguyen Van A" ← gọi
getter
print(sv.msv)        # "SV001"
# sv.ten = "B"       # AttributeError! (không có
setter)
```

```
# KHÔNG CÓ @property:
print(sv.__ten)
# → AttributeError!
# Thuộc tính private, không truy cập được

print(sv._SinhVien__ten)
# → "Nguyen Van A"
# Name mangling – XẤU, phá vỡ đóng gói!

# CÓ @property:
print(sv.ten)
# → "Nguyen Van A"
# Truy cập tự nhiên, an toàn!

# @property = cầu nối giữa
# private (ẩn) và public (dùng)
```

➡ @property (chỉ getter) = cho phép ĐỌC thuộc tính private mà không cho THAY ĐỔI. Đơn giản nhất!

# Tổng kết — 4 tính chất OOP đã hoàn thiện!



**Trừu tượng hóa**

`class, ABC, @abstractmethod`

Tuần 3



**Đóng gói**

`__private, @property, getter/setter`

Tuần 4



**Kế thừa**

`class Con(Cha), super(), override`

Tuần 5



**Đa hình**

5 dạng đa hình, ABC, Duck typing  
Magic methods, Operator overloading

Tuần 6 ★

Bạn đã có đủ kiến thức OOP — từ tuần 7 sẽ áp dụng vào File I/O nâng cao, UML và Thiết kế!

---

# **OOP07:**

- File I/O · Kiểu Dữ Liệu
  - Tuần Tự Hóa · OOP Ứng Dụng
- 
-

---

# Nội dung

1. File I/O & Kiểu dữ liệu
2. Tuần Tự Hóa Đối Tượng
3. OOP Ứng Dụng



# Vấn Đề Thực Tế

Hệ thống quản lý cán bộ: tắt chương trình → mất hết dữ liệu



*Bạn chạy chương trình QLCB, thêm 50 cán bộ, tính lương xong... rồi tắt máy.  
Mở lại — toàn bộ dữ liệu biến mất. Giải quyết thế nào?*



## Không lưu gì

Mỗi lần chạy lại nhập từ đầu.  
→ Không thực tế



## Lưu vào File Text

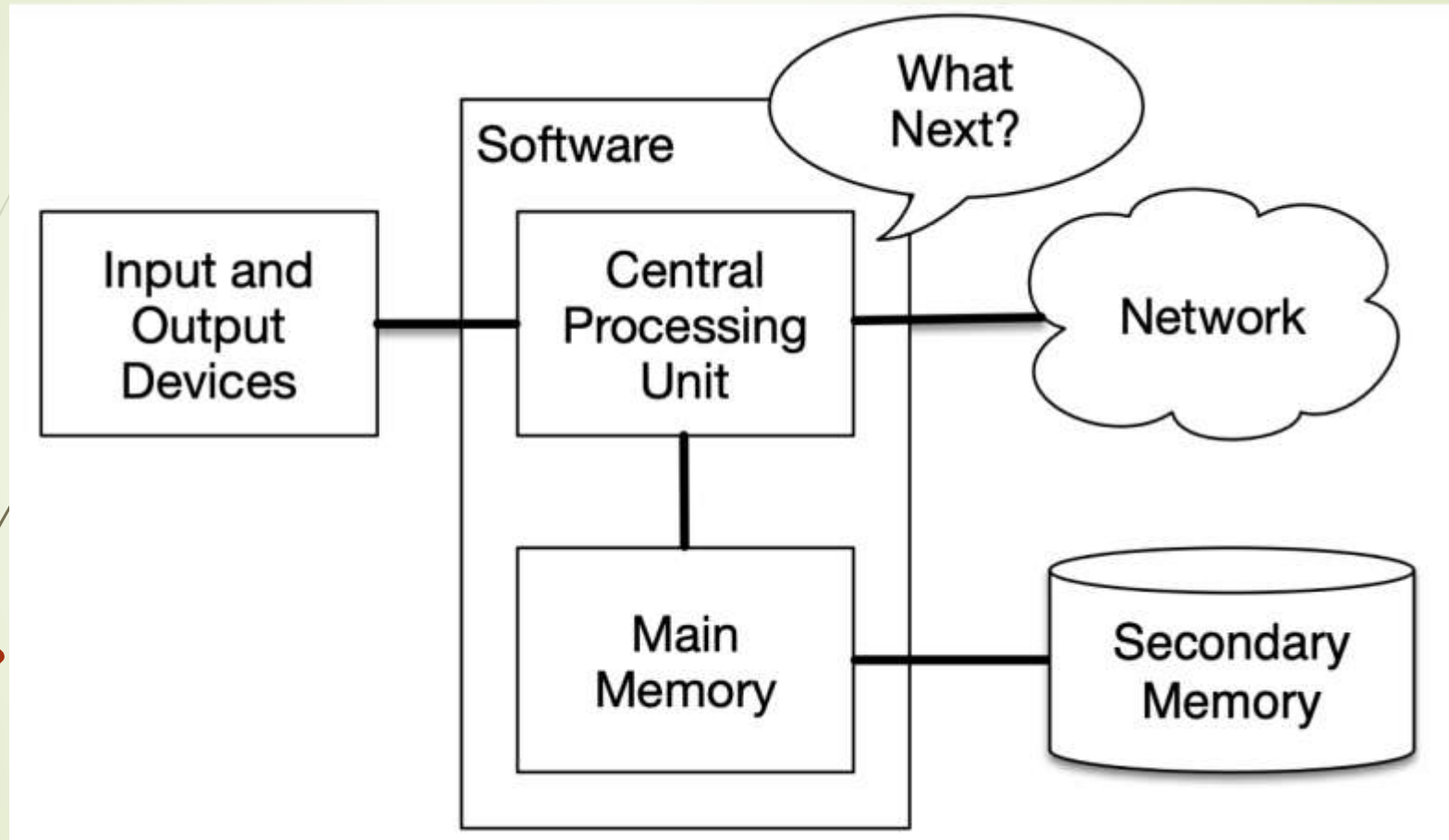
Ghi ra file .txt hoặc .csv.  
→ Đơn giản, portable, nhưng cần  
tự parse



## Tuần tự hóa đối tượng

Lưu nguyên cấu trúc Python.  
→ JSON (readable) hoặc Pickle  
(native)

# Bộ nhớ thứ cấp (Secondary Memory)



Bộ nhớ thứ cấp (Secondary Memory)

---

# 1

## Vào ra tệp tin

*File I/O*



# Hàm open() & Các Mode

Cổng giao tiếp với bộ nhớ thứ cấp

| Mode | Ý nghĩa                    | Ghi chú                    |
|------|----------------------------|----------------------------|
| 'r'  | Đọc — mặc định             | Lỗi nếu file không tồn tại |
| 'w'  | Ghi — tạo mới hoặc ghi đè  | ⚠ Xóa nội dung cũ          |
| 'a'  | Ghi thêm vào cuối          | Không xóa nội dung cũ      |
| 'x'  | Tạo mới                    | Lỗi nếu file đã tồn tại    |
| 'b'  | Nhị phân (kết hợp: rb, wb) | Dùng cho Pickle, ảnh...    |
| '+'  | Đọc + ghi đồng thời        | r+ không xóa, w+ xóa trước |

python

```
# ❌ Không an toàn
f = open("canbo.txt", "r")
data = f.read()
f.close()           # dễ quên!

# ✅ Khuyến nghị: context manager
with open("canbo.txt", "r",
          encoding="utf-8") as f:
    data = f.read()
# f tự đóng khi ra khỏi with

# ✅ Xử lý lỗi đúng cách
try:
    with open("canbo.txt", "r",
              encoding="utf-8") as f:
        data = f.read()
except FileNotFoundError:
    print("Chưa có dữ liệu lưu!")
```

# Phương Thức Đọc / Ghi

Chọn đúng phương thức cho đúng bài toán

## **read()**

Đọc toàn bộ nội dung → một chuỗi lớn

## **readlines()**

Đọc tất cả dòng → trả về list các dòng

## **writelines(lst)**

Ghi list chuỗi; không tự thêm \n

## **tell()**

Vị trí con trỏ hiện tại (byte)

## **readline()**

Đọc một dòng → dùng trong vòng lặp

## **write(s)**

Ghi chuỗi s; tự thêm \n nếu cần

## **seek(n)**

Di chuyển con trỏ về vị trí n (byte)

## **close()**

Đóng file — with tự làm việc này

---

# 2

## **Kiểu dữ liệu**

*Data Structure (List, Tuples, Dict)*

# List · Tuple · Dict - Chọn Đúng Kiểu

Ba cấu trúc dữ liệu nền tảng — mỗi cái một vai trò

## List [ ]

**Khai báo** `canbo_list = []`

**Mutable?** ☒ Thêm/sửa/xóa

**Ordered?** ☒ Có chỉ số (index)

**Hay dùng** `append()`, `pop()`, `sorted()`

**Dùng khi** Danh sách cán bộ cần thêm/xóa

## Tuple ( )

**Khai báo** `point = (x, y)`

**Mutable?** ☒ Bất biến sau khi tạo

**Ordered?** ☒ Có chỉ số (index)

**Hay dùng** Unpacking: `a, b = point`

**Dùng khi** Tọa độ, ngày sinh, hằng số

## Dict { }

**Khai báo** `{"MCC001": canbo_obj}`

**Mutable?** ☒ Thêm/sửa/xóa key

**Ordered?** ☒ (Python 3.7+)

**Hay dùng** `items()`, `get()`, `in`

**Dùng khi** Tra cứu nhanh theo mã CB

# Dict Trong Bài Quản Lý Cán Bộ

Tra cứu theo mã cán bộ — nhanh hơn dùng list rất nhiều

python

```
# Dùng List — tìm theo mã: O(n)
canbo_list = [CanBo("CB001", "An", 30, "Nam", "Hà Nội"),
               CanBo("CB002", "Bình", 25, "Nam", "TP.HCM", 7)]

def tim_theo_ma(ma):
    for cb in canbo_list:
        if cb.ho_ten == ma:
            return cb
    return None

# Dùng Dict — tìm theo mã: O(1)
canbo_dict = {
    "CB001": CanBo("CB001", "An", 30, "Nam", "Hà Nội"),
    "CB002": CanBo("CB002", "Bình", 25, "Nam", "TP.HCM", 7),
}

# Tra cứu tức thì
cb = canbo_dict.get("CB001")
if cb:
    print(cb)

# Thêm mới
canbo_dict["CB003"] = CanBo("CB003", "Châu", 28, "Nữ", "Đà Nẵng",
                             "CNTT")

# Xóa
del canbo_dict["CB001"]
```

## Khi nào dùng List?

- Cần duyệt tuần tự toàn bộ
- Thứ tự quan trọng (xếp hạng, in danh sách)
- Không cần tra cứu theo key

## Khi nào dùng Dict?

- Cần tìm/sửa/xóa theo mã (key)
- Dữ liệu có cấu trúc key→value
- Tương ứng 1-1 với JSON

# Các hàm tích hợp trong List và Tuples

- Các hàm tích hợp sẵn sau đây có thể được sử dụng với **List** và **Tuple**.

|                 |                                                                                                                                                                               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>len()</b>    | Trả về số phần tử trong list/tuple                                                                                                                                            |
| <b>max()</b>    | Nếu list/tuple chứa các <b>số</b> , hàm trả về số lớn nhất. Nếu list/tuple chứa các <b>chuỗi</b> , hàm trả về chuỗi cuối cùng theo thứ tự alphabet.                           |
| <b>min()</b>    | Nếu danh sách/tuple chứa các <b>số</b> , thì số nhỏ nhất sẽ được trả về. Nếu danh sách/tuple chứa các <b>chuỗi</b> , thì chuỗi đầu tiên trong thứ tự alphabet sẽ được trả về. |
| <b>sum()</b>    | Trả về tổng các phần tử trong list/tuple                                                                                                                                      |
| <b>sorted()</b> | Sắp xếp các phần tử trong list/tuple                                                                                                                                          |

# Các hàm tích hợp trong List

- Lớp **list** tích hợp sẵn có các phương thức sau để thực hiện các hoạt động khác nhau trên đối tượng **list**. Các phương thức sau cho phép các mục mới được thêm vào **list**.

|                 |                                                                     |
|-----------------|---------------------------------------------------------------------|
| <b>append()</b> | Thêm một đối tượng vào cuối danh sách                               |
| <b>copy()</b>   | Tạo một bản sao chép nông (shallow copy)                            |
| <b>count()</b>  | Trả về số lần xuất hiện của giá trị trong danh sách.                |
| <b>extend()</b> | Mở rộng danh sách bằng cách thêm các phần tử từ một list/tuple khác |
| <b>insert()</b> | Chèn đối tượng vào trong danh sách trước chỉ số được chỉ định.      |

# Các hàm tích hợp trong List

- Các phương thức sau để xóa bỏ các mục khỏi **list**

|                 |                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>pop()</b>    | Xóa và trả về mục tại chỉ số được cho. Nếu <b>list</b> rỗng hoặc chỉ số nằm ngoài phạm vi, ngoại lệ <code>IndexError</code> sẽ được ném ra.       |
| <b>remove()</b> | Xóa lần xuất hiện đầu tiên của giá trị trong <b>list</b> . Nếu giá trị không có trong danh sách, ngoại lệ <code>ValueError</code> sẽ được ném ra. |
| <b>clear()</b>  | Xóa tất cả các phần tử trong <b>list</b> .                                                                                                        |

- Các phương thức sau sắp xếp lại thứ tự các phần tử trong **list**

|                  |                                        |
|------------------|----------------------------------------|
| <b>reverse()</b> | Đảo ngược danh sách.                   |
| <b>sort()</b>    | Sắp xếp theo thứ tự tăng hoặc giảm dần |



# Dict

- Một kiểu dữ liệu được tích hợp sẵn trong Python được gọi là từ điển (**dictionary**). Từ điển là một trong những tính năng tốt nhất của Python; chúng là khối xây dựng cho nhiều thuật toán hiệu quả và thanh lịch (efficient and elegant algorithms).
- Từ điển (**dictionary**) của Python cũng là một kiểu đối tượng thuộc loại tập hợp (collection). Tuy nhiên, khác với danh sách (**list**) hoặc bộ (**tuple**), nó không phải là một chuỗi (**sequence**) mà là một đối tượng thuộc loại ánh xạ (**mapping**).

# Dict

- Từ điển là một tập hợp không tuần tự (unordered) của các cặp **key-value** trị được phân tách bằng dấu phẩy. **key** được ánh xạ đến **value** của nó thông qua cơ chế băm (hashing) đặc biệt của trình thông dịch Python. Sự liên kết giữa **key** và **value** được đánh dấu bằng dấu hai chấm (:) giữa chúng.

# Các hàm tích hợp trong Dict

---

|              |                                                                                                                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>len()</b> | Trả về cặp <b>key-value</b> trong <b>dict</b>                                                                                                                                                                  |
| <b>max()</b> | Nếu tất cả các <b>key</b> trong <b>dict</b> đều là số, thì số lớn nhất sẽ được trả về. Nếu tất cả các <b>key</b> trong <b>dict</b> đều là chuỗi, thì <b>key</b> cuối cùng trong thứ tự chữ cái sẽ được trả về. |
| <b>min()</b> | Nếu tất cả các <b>key</b> trong <b>dict</b> đều là số, thì số nhỏ nhất sẽ được trả về. Nếu tất cả các <b>key</b> trong <b>dict</b> đều là chuỗi, thì <b>key</b> đầu tiên trong thứ tự chữ cái sẽ được trả về.  |

# Các phương thức của lớp Dict

---

|                     |                                                                                           |
|---------------------|-------------------------------------------------------------------------------------------|
| <b>dict()</b>       | creates a new dictionary object                                                           |
| <b>clear()</b>      | Remove all items from dictionary.                                                         |
| <b>copy()</b>       | a shallow copy of dictionary                                                              |
| <b>fromkeys()</b>   | Returns a new dict with keys from iterable and values equal to value.                     |
| <b>get()</b>        | returns value associated with key                                                         |
| <b>items()</b>      | list of tuples – each tuple is key value pair                                             |
| <b>keys()</b>       | list of dictionary keys                                                                   |
| <b>pop()</b>        | remove specified key and return the corresponding value.                                  |
| <b>popitem()</b>    | remove and return some (key, value) pair as a 2-tuple.                                    |
| <b>setdefault()</b> | dictionary.get(k,dictionary), also set<br>DICTIONARY[k]=dictionary if k not in dictionary |
| <b>update()</b>     | Update dictionary from another dict/iterable                                              |
| <b>values()</b>     | list of dictionary values                                                                 |

# Các phương thức của lớp Dict

---

|                     |                                                                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>dict()</b>       | Tạo một đối tượng từ <b>dict</b>                                                                                                    |
| <b>clear()</b>      | Xóa tất cả các phần tử trong <b>dict</b>                                                                                            |
| <b>copy()</b>       | Sau chép nông (shallow copy) một <b>dict</b>                                                                                        |
| <b>fromkeys()</b>   | Trả về một từ điển mới với các <b>key</b> được lấy từ đối tượng có thể lặp và các <b>value</b> bằng với <b>value</b> được chỉ định. |
| <b>get()</b>        | Trả về giá trị liên quan đến <b>key</b> được chỉ định.                                                                              |
| <b>items()</b>      | Danh sách các bộ (tuples) – mỗi bộ là 1 cặp <b>key-value</b>                                                                        |
| <b>keys()</b>       | Danh sách <b>key</b> của <b>dict</b>                                                                                                |
| <b>pop()</b>        | Loại bỏ <b>key</b> cụ thể và trả về <b>value</b> tương ứng                                                                          |
| <b>popitem()</b>    | Loại bỏ và trả về 1 cặp <b>key-value</b> dưới dạng một bộ 2 phần tử.                                                                |
| <b>setdefault()</b> | dictionary.get(k,dictionary), cũng có thể đặt<br>DICTIONARY[k]=dictionary nếu <b>key</b> k không có trong <b>dict</b>               |
| <b>update()</b>     | Cập nhật từ điển từ một từ điển/đối tượng có thể lặp khác.                                                                          |
| <b>values()</b>     | Danh sách <b>value</b> của <b>dict</b>                                                                                              |

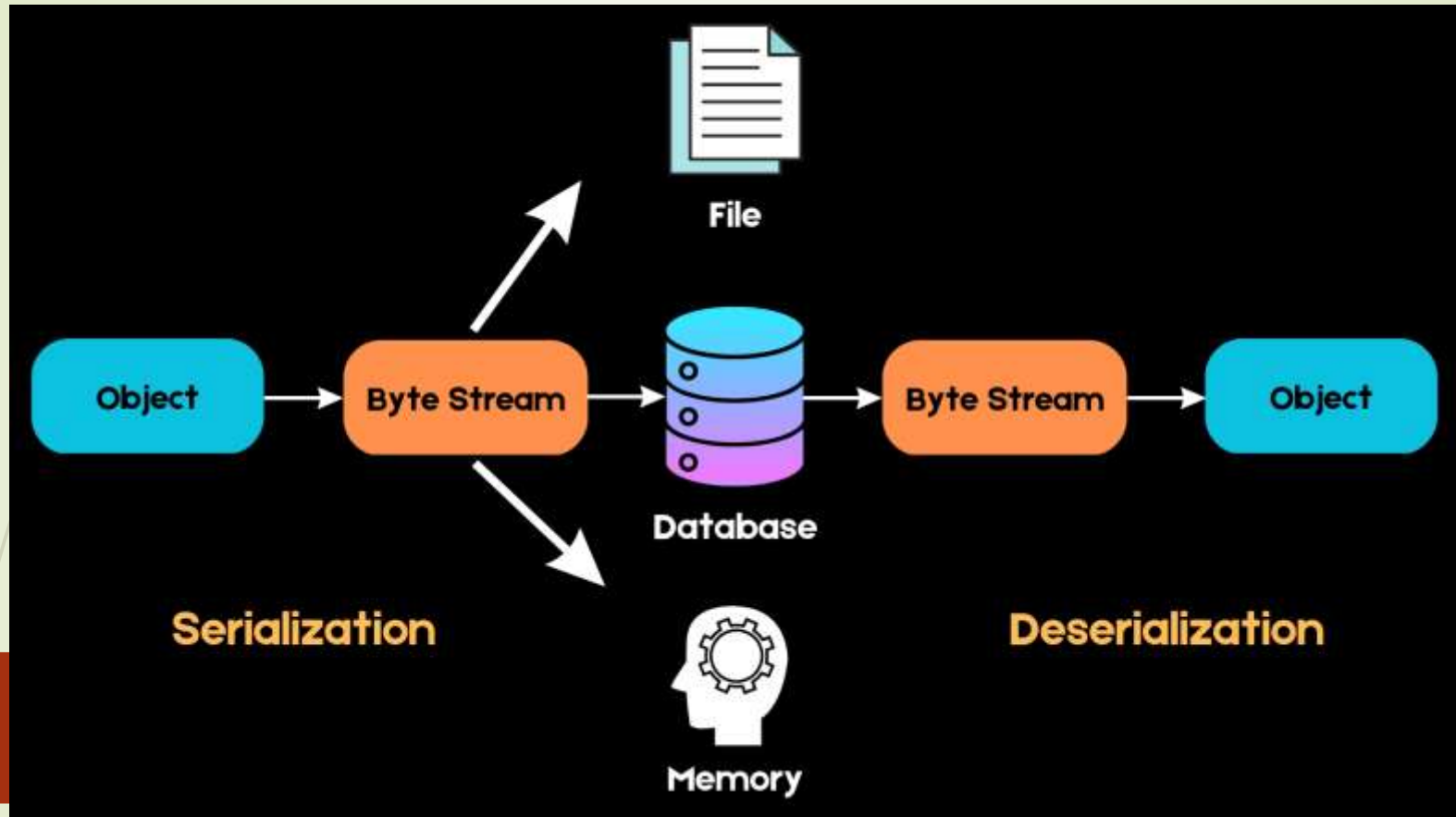
---

# 3

## Tuần tự hóa đối tượng

*Serialization - JSON · Pickle*

# Object Serialization - Deserialization



## JSON

Văn bản, đọc được bằng mắt  
Portable: Python, JS, Java đều đọc được  
Hạn chế: chỉ lưu dict/list/string/number

## Pickle

Nhị phân (binary), nhanh hơn JSON  
Lưu bất kỳ đối tượng Python nào  
Hạn chế: chỉ dùng được trong Python

# JSON — Lưu/Tải Danh Sách Cán Bộ

Chuyển đối tượng → dict, rồi dump ra file

python

```
import json

class CanBo:
    def __init__(self, ho_ten, tuoi,
                  gioi_tinh, dia_chi):
        self.ho_ten = ho_ten
        self.tuoi = tuoi
        self.gioi_tinh = gioi_tinh
        self.dia_chi = dia_chi

    def to_dict(self):          # Bước quan trọng!
        return {
            "ho_ten": self.ho_ten,
            "tuoi": self.tuoi,
            "gioi_tinh": self.gioi_tinh,
            "dia_chi": self.dia_chi,
            "loai": self.__class__.__name__,
        }

    @classmethod
    def from_dict(cls, d):     # Phục hồi từ dict
        return cls(d["ho_ten"], d["tuoi"],
                   d["gioi_tinh"], d["dia_chi"])
```

python

```
# === LƯU ===
danh_sach = [
    CanBo("Nguyễn An", 30, "Nam", "Hà Nội"),
    CongNhan("Trần Bình", 25, "Nam", "TP.HCM", 5),
]

data = [cb.to_dict() for cb in danh_sach]

with open("canbo.json", "w",
          encoding="utf-8") as f:
    json.dump(data, f,
              ensure_ascii=False, indent=2)

# === TẢI ===
with open("canbo.json", "r",
          encoding="utf-8") as f:
    raw = json.load(f)

# Khôi phục đúng loại theo "loai"
ds_loaded = [CanBo.from_dict(d)
              for d in raw]

for cb in ds_loaded:
    print(cb)
```



# Pickle — Lưu Nguyên Đối Tượng Python

Không cần to\_dict() — nhưng chỉ dùng được trong Python

python

```
import pickle

# === LƯU (serialize) ===
danh_sach = [
    CanBo("Nguyễn An", 30, "Nam", "Hà Nội"),
    CongNhan("Trần Bình", 25, "Nam", "TP.HCM", 5),
    KySu("Lê Châu", 28, "Nữ", "Đà Nẵng", "CNTT"),
    NhanVien("Phạm Dũng", 35, "Nam",
             "Cần Thơ", "Kế toán"),
]

with open("canbo.pkl", "wb") as f: # wb: binary
    pickle.dump(danh_sach, f)

print("Đã lưu", len(danh_sach), "cán bộ")
```

python

```
# === TẢI (deserialize) ===
try:
    with open("canbo.pkl", "rb") as f:
        ds = pickle.load(f)

    print(f"Tải {len(ds)} cán bộ thành công")
    for cb in ds:
        print(cb) # gọi __str__
        print(cb.tinh_luong()) # method vẫn hoạt động!

except FileNotFoundError:
    print("Chưa có dữ liệu — khởi tạo mới")
    ds = []
```

|                        | JSON | Pickle        |
|------------------------|------|---------------|
| Đọc được bằng mắt      | ✓    | ✗             |
| Cần to_dict()?         | ✓ Có | ✗ Không       |
| Ngôn ngữ khác đọc được | ✓    | ✗ Python only |
| Lưu method của class   | ✗    | ✓             |

# Hệ Thống QLCB Có Lưu Trữ

Tích hợp toàn bộ: class · ngoại lệ · dict · JSON

python

```
def _luu_du_lieu(self):
    data = [cb.to_dict()
             for cb in self.ds.values()]
    with open(self.FILE, "w",
              encoding="utf-8") as f:
        json.dump(data, f,
                  ensure_ascii=False, indent=2)

def _tai_du_lieu(self):
    try:
        with open(self.FILE, "r",
                  encoding="utf-8") as f:
            for d in json.load(f):
                cls = LOAI_MAP.get(
                    d["loai"], CanBo)
                cb = cls.from_dict(d)
                self.ds[cb.ho_ten] = cb
    except FileNotFoundError:
        pass # lần đầu chạy
```

---

# 4



**Ứng dụng OOP – Xây dựng  
hệ thống hoàn chỉnh**

# Kiến Trúc Hệ Thống QLCB — Tổng Quan

Trước khi demo: hiểu rõ vai trò từng lớp

Hệ thống được tổ chức theo nguyên tắc: mỗi lớp một trách nhiệm duy nhất

## TẦNG MODEL

### CanBo

Lớp cha — `ho_ten`, `tuoi`, `gioi_tinh`, `dia_chi`; `__str__`;  `tinh_luong()`; `to_dict()`

### CongNhan

Kế thừa CanBo — thêm: `bac` (1–10);  `tinh_luong()` = `bac` × `HE_SO_BAC`

### KySu

Kế thừa CanBo — thêm: `nganh_dao_tao`;  `tinh_luong()` theo hệ số ngành

### NhanVien

Kế thừa CanBo — thêm: `cong_viec`; mô tả công việc qua `__str__()`

## TẦNG LOGIC

### QuanLyCanBo

Điều phối toàn bộ: `them()`, `xoa()`, `tim()`, `_luu_du_lieu()`, `_tai_du_lieu()`

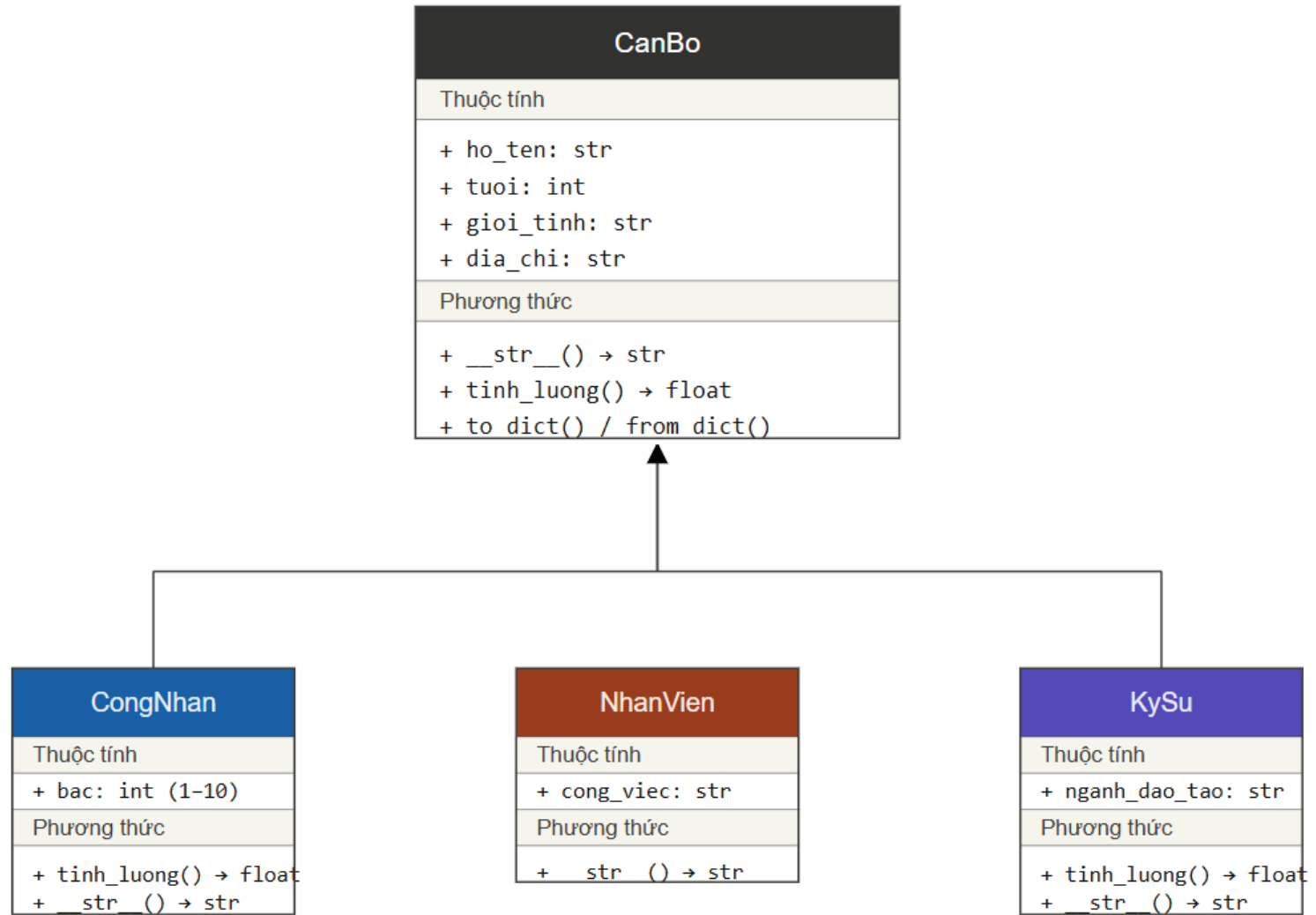
## TẦNG I/O

### json / pickle

Tuần tự hóa: `dump/load` — tách biệt khỏi logic nghiệp vụ

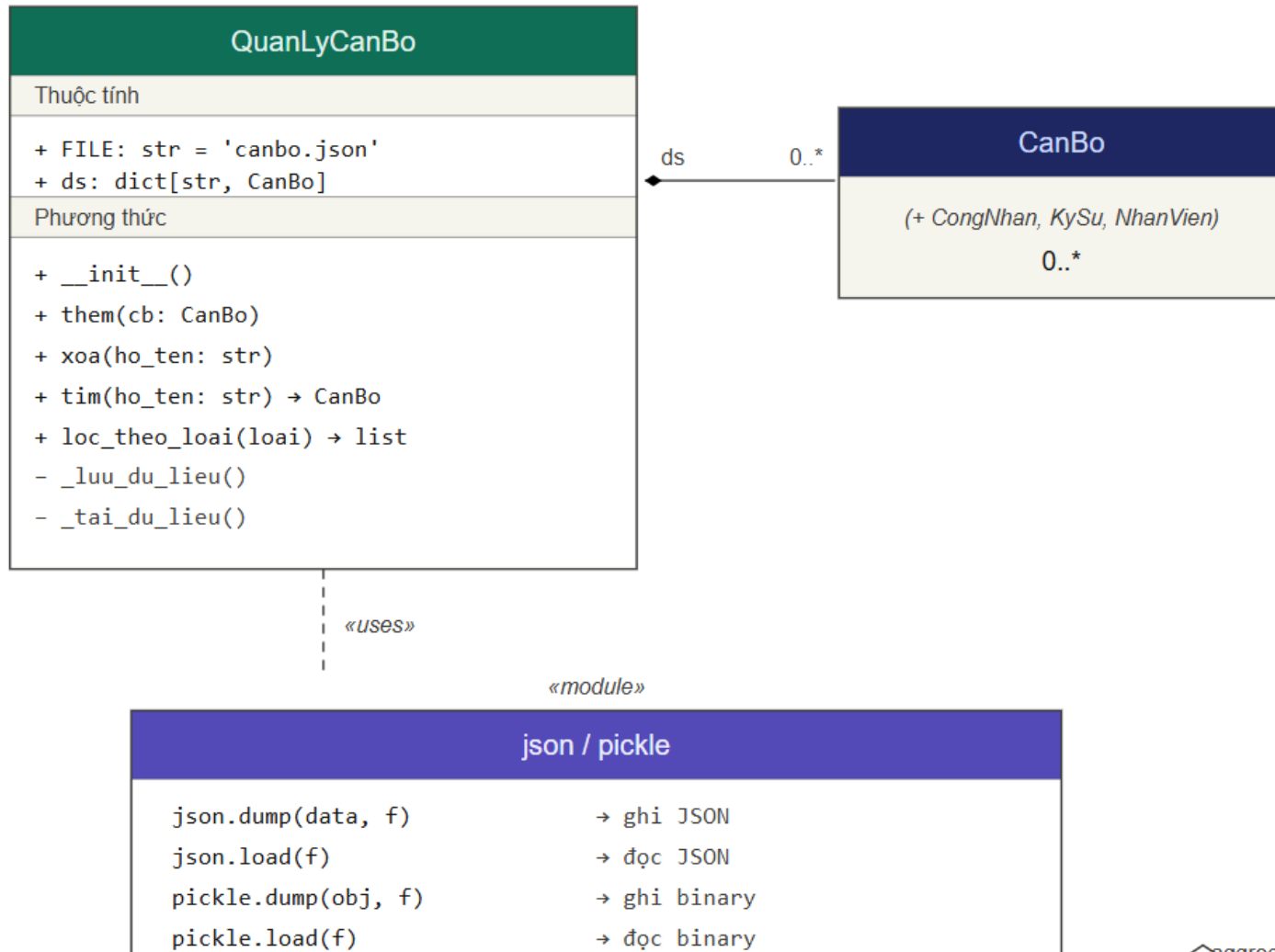
# Class Diagram — Tầng Model

CanBo · CongNhan · KySu — quan hệ kế thừa và thuộc tính



# Class Diagram — Tầng Logic & I/O

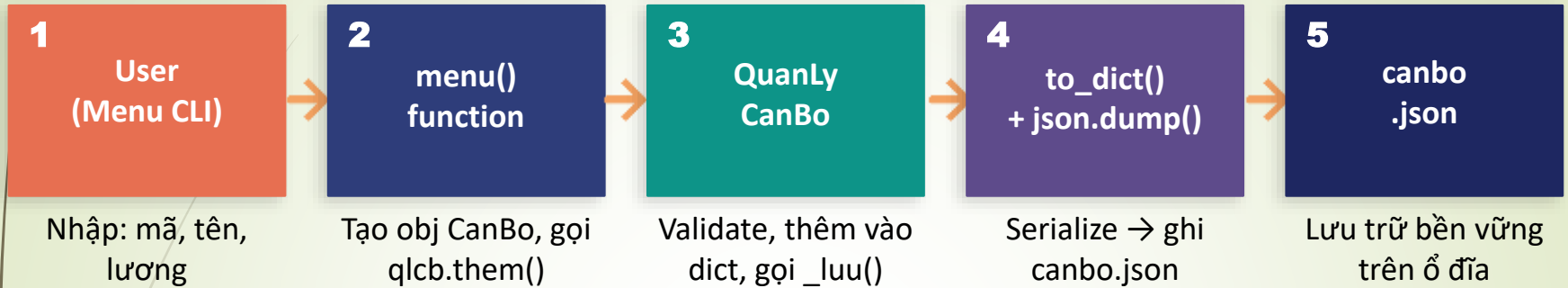
QuanLyCanBo điều phối toàn bộ — I/O layer tách biệt



◊ aggregation  
- dependency

# Luồng Dữ Liệu — Từ Thao Tác Đến File

Trace từng bước: user gõ 'Thêm cán bộ' → ghi JSON



← Chiều ngược: json.load() → from\_dict() → dict[ma] → hiển thị cho user

## Trace chi tiết: qlcb.them(CongNhan('Lan', 25, 'Nữ', 'Hà Nội', 3))

- 1. CanBo.\_\_init\_\_() → ho\_ten='Lan', tuoi=25, gioi\_tinh='Nữ', dia\_chi='Hà Nội'
- 2. CongNhan.\_\_init\_\_() → bac=3 (bậc công nhân)
- 3. QuanLyCanBo.them() → self.ds['Lan'] = <CongNhan object> → \_luu()
- 4. json.dump([cb.to\_dict() for cb in ds.values()], f) → ghi canbo.json

# Mỗi Lớp — Một Trách Nhiệm

*Single Responsibility: thiết kế tốt trước khi biết tên nó*

Nếu mô tả trách nhiệm của một lớp cần dùng chữ "và" — đó là dấu hiệu lớp đang làm quá nhiều việc.

CanBo / CongNhan / KySu

Biết về bản thân mình: thuộc tính và cách tính lương

✓ `__str__`,  `tinh_luong()`, `to_dict()`

✗ Không biết về file, không biết về menu

QuanLyCanBo

Biết về tập hợp cán bộ: thêm, xóa, tìm, lưu, tải

✓ `them()`, `xoa()`, `tim()`, `_luu()`, `_tai()`

✗ Không biết cách hiển thị menu, không biết format JSON cụ thể

`menu()` function

Biết về giao tiếp với người dùng: in, nhập, điều hướng

✓ Nhận input, in output, gọi `QuanLyCanBo`

✗ Không tự thêm/xóa dữ liệu trực tiếp

`json` module

Biết về định dạng JSON: đọc và ghi

✓ `dump()`, `load()` — chuẩn thư viện Python

✗ Không biết về logic nghiệp vụ



# Menu CLI: Mọi Thứ Hoạt Động Cùng Nhau

*Bức tranh OOP hoàn chỉnh*

python

```
def menu():
    qlcb = QuanLyCanBo() # tự tải JSON khi khởi tạo
    while True:
        print("\n===== QUẢN LÝ CÁN BỘ =====")
        print("1. Xem danh sách toàn bộ")
        print("2. Thêm Công Nhân")
        print("3. Thêm Kỹ Sư")
        print("4. Thêm Nhân Viên")
        print("5. Tìm theo họ tên")
        print("6. Xóa cán bộ")
        print("0. Thoát")
        chon = input("Chọn: ").strip()

        if chon == "1":
            for cb in qlcb.ds.values():
                print(cb) # gọi __str__
        elif chon == "2":
            ten = input("Họ tên: ")
            t = int(input("Tuổi: "))
            gt = input("Giới tính: ")
            dc = input("Địa chỉ: ")
            bac = int(input("Bậc (1-10): "))
            try:
                qlcb.them(CongNhan(ten,t,gt,dc,bac))
                print("Đã thêm và lưu!")
            except ValueError as e:
                print("Lỗi:", e)
```

python

```
        elif chon == "3":
            ten = input("Họ tên: ")
            t,gt,dc = int(input("Tuổi: ")),input("Giới tính: "),input("Địa chỉ: ")
            ngành = input("Ngành đào tạo: ")
            try:
                qlcb.them(KySu(ten,t,gt,dc,nganh))
                print("Đã thêm và lưu!")
            except ValueError as e:
                print("Lỗi:", e)
        elif chon == "5":
            cb = qlcb.tim(input("Họ tên: "))
            print(cb if cb else "Không tìm thấy")
        elif chon == "6":
            try:
                qlcb.xoa(input("Họ tên cần xóa: "))
                print("Đã xóa và lưu!")
            except KeyError as e:
                print("Lỗi:", e)
        elif chon == "0":
            break

if __name__ == "__main__":
    menu()
```

# Tất Cả OOP Trong Một Hệ Thống

Nhìn lại — mọi kỹ thuật đã học đều có mặt ở đây



## Lớp & Kế thừa

CanBo ← CongNhan, KySu, NhanVien  
Lớp con mở rộng không sửa lớp cha



## Đa hình & Magic Methods

`__str__` in thông tin đúng theo từng loại  
`__call__` cho kết quả khác nhau

## Ngoại lệ

raise `ValueError`, `KeyError`  
try/except kiểm soát luồng lỗi



## File I/O

`open()` với context manager  
encoding utf-8 cho tiếng Việt



## Dict

Tra cứu  $O(1)$  theo họ tên  
Key = họ\_tên, Value = đối tượng CB



## Serialization

`to_dict()` / `from_dict()`  
`json.dump()` / `json.load()`

# Câu Hỏi Mở?

---

## Hệ thống tốt hơn nghĩa là gì?

? Nếu thêm loại *NhanVienHopDong* mới, phải sửa bao nhiêu chỗ trong code?

? Nếu muốn đổi từ *JSON* sang *SQLite*, cần sửa những file nào?

? Nếu cần thêm tính năng "xuất báo cáo *PDF*", đặt code đó ở đâu?

? Khi nào thì nên tách *QuanLyCanBo* thành 2 class nhỏ hơn?

# Tuần 8 — Thiết Kế Hệ Thống & UML

*Từ code sang tư duy kiến trúc phần mềm*

*Câu hỏi vừa đặt ra ở Tiết 4 chính là vấn đề mà Thiết kế hướng đối tượng giải quyết.*

## **Class Diagram (UML)**

Vẽ được sơ đồ lớp đầy đủ

Quan hệ: kế thừa, kết hợp, phụ thuộc

Multiplicity: 1..\*, 0..1

## **SOLID Principles**

Single Responsibility — một lớp một việc

Open/Closed — mở rộng, không sửa

Dependency Inversion — phụ thuộc vào trừu tượng

## **Refactor Từ Diagram**

Đọc diagram → viết code

Phát hiện design smell từ diagram

Tái cấu trúc hệ thống cán bộ

## **Case Study Tổng Hợp**

Phân tích requirement → UML

Code theo diagram → So sánh trước/sau

Chuẩn bị cho bài tập lớn

# Bài tập về nhà

Thực hành File I/O · Dict · Serialization trên bài cán bộ

Dựa trên bài cán bộ đã xây dựng ở chương 5 (CanBo, CongNhan, KySu, NhanVien), hoàn thiện hệ thống theo các yêu cầu sau:

1. Đọc dữ liệu từ file **canbo.csv** (họ tên, tuổi, giới tính, địa chỉ, loại, bậc/ngành/công việc) vào danh sách đối tượng. Xử lý **FileNotFoundError**, **ValueError** đúng cách.
2. Lưu danh sách vào **dict** với key là họ tên. Cài đặt các thao tác: thêm, xóa, tìm theo họ tên, tìm theo loại (CongNhan/KySu/NhanVien). In ra 3 cán bộ có bậc/lương cao nhất.
3. Thêm **to\_dict()** và **from\_dict()** vào các lớp. Lưu/tải toàn bộ danh sách ra file **canbo.json** (ensure\_ascii=False). Đảm bảo khi tải lại, đúng loại đối tượng được khôi phục theo trường "loai".
4. ★ Đóng gói toàn bộ vào lớp **QuanLyCanBo** với menu CLI — tự động lưu JSON sau mỗi thao tác, xử lý mọi ngoại lệ, chương trình không được crash.

# Tổng kết

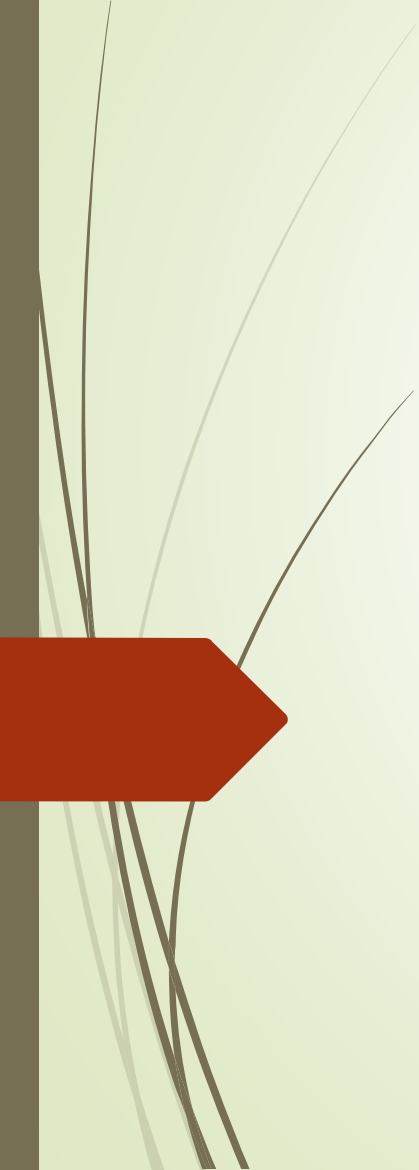
---

- ✓ Tiết 1 — Review: Đa hình · Magic methods · Ngoại lệ · Code review Tuần 6
- ✓ Tiết 2 — File I/O: open(), read/write, context manager; List · Dict · Tuple
- ✓ Tiết 3 — Serialization: to\_dict/from\_dict, JSON, Pickle → lưu/tải danh sách cán bộ
- ✓ Tiết 4 — Menu CLI hoàn chỉnh: tất cả OOP hoạt động cùng nhau trong một hệ thống



***Tuần 8 → Thiết Kế Hệ Thống & UML***

---

A decorative graphic on the left side of the slide featuring several thin, curved lines representing grass or reeds. At the bottom left, there is a solid red arrow pointing to the right.

# Thank you!

*Any questions?*